

C Data Structures & Algorithms

Easy • Medium • Heavy — Programs with Sample Input & Output

This PDF contains 17 exam-oriented C programs covering recursion, arrays, stack, queue, linked list, searching, sorting, BST, graph traversal, and single-source shortest path using Dijkstra's algorithm.

EASY

1. Recursion – Factorial

C Code

```
#include <stdio.h>

long long factorial(int n) {
    if (n <= 1)
        return 1;
    return n * factorial(n - 1);
}

int main() {
    int n;
    printf("Enter a number: ");
    scanf("%d", &n);
    printf("Factorial = %lld\n", factorial(n));
    return 0;
}
```

Sample Input / Output

```
Enter a number: 5
Factorial = 120
```

2. Recursion – Fibonacci

C Code

```
#include <stdio.h>

int fibonacci(int n) {
    if (n <= 1)
        return n;
    return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
    int n, i;
    printf("Enter number of terms: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
        printf("%d ", fibonacci(i));

    return 0;
}
```

Sample Input / Output

```
Enter number of terms: 7
0 1 1 2 3 5 8
```

3. String Operations – Length, Reverse, Copy, Compare

C Code

```
#include <stdio.h>
#include <string.h>

int main() {
    char s1[100], s2[100], copy[100], rev[100];
    int i, len;

    printf("Enter first string: ");
    scanf("%99s", s1);
    printf("Enter second string: ");
    scanf("%99s", s2);

    len = strlen(s1);
    strcpy(copy, s1);

    for (i = 0; i < len; i++)
        rev[i] = s1[len - 1 - i];
```

```

    rev[len] = '\0';

    printf("Length = %d\n", len);
    printf("Copy = %s\n", copy);
    printf("Reverse = %s\n", rev);

    if (strcmp(s1, s2) == 0)
        printf("Strings are equal\n");
    else
        printf("Strings are not equal\n");

    return 0;
}

```

Sample Input / Output

```

Enter first string: hello
Enter second string: world
Length = 5
Copy = hello
Reverse = olleh
Strings are not equal

```

4. Array Operations – Insert, Delete, Display

C Code

```

#include <stdio.h>

int main() {
    int a[100], n, i, pos, value;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Enter position and value to insert: ");
    scanf("%d %d", &pos, &value);

    for (i = n; i >= pos; i--)
        a[i] = a[i - 1];
    a[pos - 1] = value;
    n++;

    printf("After insertion: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\nEnter position to delete: ");
    scanf("%d", &pos);

    for (i = pos - 1; i < n - 1; i++)
        a[i] = a[i + 1];
    n--;

    printf("After deletion: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}

```

Sample Input / Output

```

Enter number of elements: 5
Enter elements: 10 20 30 40 50
Enter position and value to insert: 3 25
After insertion: 10 20 25 30 40 50
Enter position to delete: 4
After deletion: 10 20 25 40 50

```

MEDIUM – STACK, QUEUE & LINKED LIST

5. Stack using Array

C Code

```
#include <stdio.h>
#define MAX 100

int stack[MAX], top = -1;

void push(int x) {
    if (top == MAX - 1)
        printf("Stack Overflow\n");
    else
        stack[++top] = x;
}

void pop() {
    if (top == -1)
        printf("Stack Underflow\n");
    else
        printf("Deleted element: %d\n", stack[top--]);
}

void display() {
    int i;
    if (top == -1) {
        printf("Stack is empty\n");
        return;
    }
    printf("Stack: ");
    for (i = top; i >= 0; i--)
        printf("%d ", stack[i]);
    printf("\n");
}

int main() {
    int choice, value;

    while (1) {
        printf("\n1.Push 2.Pop 3.Display 4.Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        if (choice == 1) {
            printf("Enter value: ");
            scanf("%d", &value);
            push(value);
        } else if (choice == 2) {
            pop();
        } else if (choice == 3) {
            display();
        } else if (choice == 4) {
            break;
        } else {
            printf("Invalid choice\n");
        }
    }
    return 0;
}
```

Sample Input / Output

```
1.Push 2.Pop 3.Display 4.Exit
Enter choice: 1
Enter value: 10
Enter choice: 1
Enter value: 20
Enter choice: 3
Stack: 20 10
Enter choice: 2
Deleted element: 20
```

6. Queue using Array

C Code

```

#include <stdio.h>
#define MAX 100

int queue[MAX], front = -1, rear = -1;

void enqueue(int x) {
    if (rear == MAX - 1) {
        printf("Queue Overflow\n");
        return;
    }
    if (front == -1)
        front = 0;
    queue[++rear] = x;
}

void dequeue() {
    if (front == -1 || front > rear) {
        printf("Queue Underflow\n");
        return;
    }
    printf("Deleted element: %d\n", queue[front++]);
    if (front > rear)
        front = rear = -1;
}

void display() {
    int i;
    if (front == -1) {
        printf("Queue is empty\n");
        return;
    }
    printf("Queue: ");
    for (i = front; i <= rear; i++)
        printf("%d ", queue[i]);
    printf("\n");
}

int main() {
    int choice, value;

    while (1) {
        printf("\n1.Enqueue 2.Dequeue 3.Display 4.Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        if (choice == 1) {
            printf("Enter value: ");
            scanf("%d", &value);
            enqueue(value);
        } else if (choice == 2) {
            dequeue();
        } else if (choice == 3) {
            display();
        } else if (choice == 4) {
            break;
        } else {
            printf("Invalid choice\n");
        }
    }
    return 0;
}

```

Sample Input / Output

```

1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter choice: 1
Enter value: 10
Enter choice: 1
Enter value: 20
Enter choice: 3
Queue: 10 20
Enter choice: 2
Deleted element: 10

```

7. Singly Linked List – Insert and Display

C Code

```

#include <stdio.h>

```

```

#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

int main() {
    struct Node *head = NULL, *newNode, *temp;
    int n, value, i;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        newNode = (struct Node *)malloc(sizeof(struct Node));
        printf("Enter value: ");
        scanf("%d", &value);

        newNode->data = value;
        newNode->next = NULL;

        if (head == NULL)
            head = newNode;
        else {
            temp = head;
            while (temp->next != NULL)
                temp = temp->next;
            temp->next = newNode;
        }
    }

    printf("Linked List: ");
    temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");

    return 0;
}

```

Sample Input / Output

```

Enter number of nodes: 4
Enter value: 10
Enter value: 20
Enter value: 30
Enter value: 40
Linked List: 10 -> 20 -> 30 -> 40 -> NULL

```

MEDIUM – SEARCHING & SORTING

8. Linear Search

C Code

```
#include <stdio.h>

int main() {
    int a[100], n, key, i, found = -1;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Enter element to search: ");
    scanf("%d", &key);

    for (i = 0; i < n; i++) {
        if (a[i] == key) {
            found = i;
            break;
        }
    }

    if (found != -1)
        printf("Element found at position %d\n", found + 1);
    else
        printf("Element not found\n");

    return 0;
}
```

Sample Input / Output

```
Enter number of elements: 5
Enter elements: 12 25 7 30 18
Enter element to search: 30
Element found at position 4
```

9. Binary Search

C Code

```
#include <stdio.h>

int main() {
    int a[100], n, key, low, high, mid, found = -1;
    int i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter sorted elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Enter element to search: ");
    scanf("%d", &key);

    low = 0;
    high = n - 1;

    while (low <= high) {
        mid = (low + high) / 2;

        if (a[mid] == key) {
            found = mid;
            break;
        } else if (key < a[mid])
            high = mid - 1;
        else
            low = mid + 1;
    }
}
```

```

    if (found != -1)
        printf("Element found at position %d\n", found + 1);
    else
        printf("Element not found\n");

    return 0;
}

```

Sample Input / Output

```

Enter number of elements: 6
Enter sorted elements: 5 10 15 20 25 30
Enter element to search: 25
Element found at position 5

```

10. Bubble Sort

C Code

```

#include <stdio.h>

int main() {
    int a[100], n, i, j, temp;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for (i = 0; i < n - 1; i++)
        for (j = 0; j < n - i - 1; j++)
            if (a[j] > a[j + 1]) {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }

    printf("Sorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}

```

Sample Input / Output

```

Enter number of elements: 5
Enter elements: 64 25 12 22 11
Sorted array: 11 12 22 25 64

```

11. Insertion Sort

C Code

```

#include <stdio.h>

int main() {
    int a[100], n, i, j, key;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for (i = 1; i < n; i++) {
        key = a[i];
        j = i - 1;

        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
}

```

```

    }

    printf("Sorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}

```

Sample Input / Output

```

Enter number of elements: 5
Enter elements: 12 11 13 5 6
Sorted array: 5 6 11 12 13

```

12. Quick Sort

C Code

```

#include <stdio.h>

void quickSort(int a[], int low, int high) {
    int i = low, j = high, pivot = a[(low + high) / 2], temp;

    while (i <= j) {
        while (a[i] < pivot)
            i++;
        while (a[j] > pivot)
            j--;

        if (i <= j) {
            temp = a[i];
            a[i] = a[j];
            a[j] = temp;
            i++;
            j--;
        }
    }

    if (low < j)
        quickSort(a, low, j);
    if (i < high)
        quickSort(a, i, high);
}

int main() {
    int a[100], n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    quickSort(a, 0, n - 1);

    printf("Sorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}

```

Sample Input / Output

```

Enter number of elements: 6
Enter elements: 10 7 8 9 1 5
Sorted array: 1 5 7 8 9 10

```

13. Merge Sort

C Code

```

#include <stdio.h>

void merge(int a[], int low, int mid, int high) {

```

```

int temp[100], i = low, j = mid + 1, k = 0;

while (i <= mid && j <= high) {
    if (a[i] < a[j])
        temp[k++] = a[i++];
    else
        temp[k++] = a[j++];
}

while (i <= mid)
    temp[k++] = a[i++];

while (j <= high)
    temp[k++] = a[j++];

for (i = low, k = 0; i <= high; i++, k++)
    a[i] = temp[k];
}

void mergeSort(int a[], int low, int high) {
    int mid;
    if (low < high) {
        mid = (low + high) / 2;
        mergeSort(a, low, mid);
        mergeSort(a, mid + 1, high);
        merge(a, low, mid, high);
    }
}

int main() {
    int a[100], n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    mergeSort(a, 0, n - 1);

    printf("Sorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}

```

Sample Input / Output

```

Enter number of elements: 6
Enter elements: 38 27 43 3 9 82
Sorted array: 3 9 27 38 43 82

```

HEAVY – TREES & GRAPHS

14. Binary Search Tree – Insertion and Traversals

C Code

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
};

struct Node* createNode(int value) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->data = value;
    node->left = node->right = NULL;
    return node;
}

struct Node* insert(struct Node* root, int value) {
    if (root == NULL)
        return createNode(value);

    if (value < root->data)
        root->left = insert(root->left, value);
    else if (value > root->data)
        root->right = insert(root->right, value);

    return root;
}

void inorder(struct Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

void preorder(struct Node* root) {
    if (root != NULL) {
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}

void postorder(struct Node* root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);
    }
}

int main() {
    struct Node* root = NULL;
    int n, value, i;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter values: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &value);
        root = insert(root, value);
    }

    printf("Inorder: ");
    inorder(root);
    printf("\nPreorder: ");
    preorder(root);
    printf("\nPostorder: ");
    postorder(root);

    return 0;
}
```

Sample Input / Output

```
Enter number of nodes: 7
Enter values: 50 30 70 20 40 60 80
Inorder: 20 30 40 50 60 70 80
Preorder: 50 30 20 40 70 60 80
Postorder: 20 40 30 60 80 70 50
```

15. Graph Traversal – BFS

C Code

```
#include <stdio.h>

int main() {
    int graph[20][20], visited[20] = {0};
    int queue[20], front = 0, rear = 0;
    int n, start, i, j, v;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &graph[i][j]);

    printf("Enter starting vertex (1-%d): ", n);
    scanf("%d", &start);
    start--;

    queue[rear++] = start;
    visited[start] = 1;

    printf("BFS: ");
    while (front < rear) {
        v = queue[front++];
        printf("%d ", v + 1);

        for (i = 0; i < n; i++) {
            if (graph[v][i] && !visited[i]) {
                queue[rear++] = i;
                visited[i] = 1;
            }
        }
    }

    return 0;
}
```

Sample Input / Output

```
Enter number of vertices: 5
Enter adjacency matrix:
0 1 1 0 0
1 0 0 1 0
1 0 0 1 1
0 1 1 0 1
0 0 1 1 0
Enter starting vertex (1-5): 1
BFS: 1 2 3 4 5
```

16. Graph Traversal – DFS

C Code

```
#include <stdio.h>

int graph[20][20], visited[20], n;

void dfs(int v) {
    int i;
    visited[v] = 1;
    printf("%d ", v + 1);

    for (i = 0; i < n; i++)
        if (graph[v][i] && !visited[i])
            dfs(i);
}
```

```

        dfs(i);
    }

int main() {
    int start, i, j;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &graph[i][j]);

    printf("Enter starting vertex (1-%d): ", n);
    scanf("%d", &start);

    printf("DFS: ");
    dfs(start - 1);

    return 0;
}

```

Sample Input / Output

```

Enter number of vertices: 5
Enter adjacency matrix:
0 1 1 0 0
1 0 0 1 0
1 0 0 1 1
0 1 1 0 1
0 0 1 1 0
Enter starting vertex (1-5): 1
DFS: 1 2 4 3 5

```

17. Single Source Shortest Path – Dijkstra

C Code

```

#include <stdio.h>
#define INF 99999

int main() {
    int cost[20][20], dist[20], visited[20] = {0};
    int n, source, i, j, count, u, min;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter cost matrix (0 for no edge):\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
            if (i != j && cost[i][j] == 0)
                cost[i][j] = INF;
        }
    }

    printf("Enter source vertex (1-%d): ", n);
    scanf("%d", &source);
    source--;

    for (i = 0; i < n; i++)
        dist[i] = cost[source][i];

    dist[source] = 0;
    visited[source] = 1;

    for (count = 1; count < n; count++) {
        min = INF;
        u = -1;

        for (i = 0; i < n; i++) {
            if (!visited[i] && dist[i] < min) {
                min = dist[i];
                u = i;
            }
        }
    }
}

```

```

        if (u == -1)
            break;

        visited[u] = 1;

        for (j = 0; j < n; j++) {
            if (!visited[j] && dist[u] + cost[u][j] < dist[j])
                dist[j] = dist[u] + cost[u][j];
        }

        printf("Shortest distances from vertex %d:\n", source + 1);
        for (i = 0; i < n; i++)
            printf("To %d = %d\n", i + 1, dist[i]);

        return 0;
    }

```

Sample Input / Output

```

Enter number of vertices: 5
Enter cost matrix (0 for no edge):
0 10 0 30 100
10 0 50 0 0
0 50 0 20 10
30 0 20 0 60
100 0 10 60 0
Enter source vertex (1-5): 1
Shortest distances from vertex 1:
To 1 = 0
To 2 = 10
To 3 = 50
To 4 = 30
To 5 = 60

```